Agora temos evidência concreta da estrutura atual através de schemas_tables.csv. [schemas_tables | Excel]

Após comparar sua visão original com o banco real, minha conclusão mudou em um ponto importante:

> O sistema hoje está implementado como um híbrido entre SaaS multi-tenant tradicional e um catálogo semântico Protheus. Porém sua visão original é significativamente mais simples e mais alinhada ao objetivo do produto. [schemas_tables | Excel]

Diagnóstico arquitetural real

O banco atualmente possui dois grandes blocos:

Schema public

Contém:

- tenants
- usuários
- contratos
- módulos
- planos
- licenciamento
- auditoria
- conversas
- dashboards
- conectores
- ambientes
- RBAC (roles/permissions) [schemas_tables | Excel]

Schema rodol_prd

Contém:

- dictionary_tables
- dictionary_fields
- dictionary_indexes
- documentos RAG
- regras de negócio
- métricas
- permissões de tabela

- permissões de campo
- snapshots de schema
- auditoria SQL [schemas_tables | Excel]

O principal problema encontrado

Sua arquitetura desejada é:

```
public
empresa_a
empresa_b
empresa_c
```

Mas o banco atual está estruturado como:

```
public
rodol_prd
```

[schemas_tables | Excel]

Isso significa que hoje:

- o dicionário de todas as empresas está centralizado
- as regras de todas as empresas estão centralizadas
- os documentos de todas as empresas estão centralizados

utilizando apenas:

```
tenant_id
company_id
```

para separação lógica. [schemas_tables | Excel]

Essa arquitetura funciona?

Sim.

Mas não é a arquitetura que você descreveu.

Hoje o isolamento depende de:

```
WHERE tenant_id = ?
```

em praticamente tudo:

- dictionary_fields
- dictionary_tables
- memories
- documents
- company_info
- tenant_schemas
- business_rules
- permissões

[schemas_tables | Excel]

Já no modelo que você propôs:

```
Schema PostgreSQL
=
Tenant
```

não existe risco de esquecer:

```
WHERE tenant_id
```

porque as tabelas estão fisicamente separadas.

Problema crítico nº 1

Duplicidade de cadastro da empresa

Hoje existem:

```
public.tenant
```

e

```
rodol_prd.company_info
```

[schemas_tables | Excel]

Ambos armazenam:

- credenciais Protheus
- URLs
- configurações do sistema
- ambiente
- informações cadastrais

[schemas_tables | Excel]

Consequência:

inevitavelmente ocorrerá divergência.

Recomendação:

Manter somente:

```
public.tenant
```

como cadastro mestre.

Problema crítico nº 2

Permissões de tabela e campo

Existem:

```
rodol_prd.tenant_table_permissions
```

e

```
rodol_prd.tenant_field_permissions
```

[schemas_tables | Excel]

Comparando com sua proposta:

isso não deveria existir.

Seu conceito é:

```
dictionary_tables
```

já representa a whitelist.

```
dictionary_fields
```

já representa a whitelist.

Logo:

```
if tabela not in dictionary:
    deny
```

Sem precisar de:

```
role
permission
table_permission
field_permission
```

Minha recomendação:

Remover

```
tenant_table_permissions

tenant_field_permissions
```

[schemas_tables | Excel]

Problema crítico nº 3

RBAC excessivamente complexo

Hoje existem:

```
roles

permissions

role_permissions

user_company_access
```

[schemas_tables | Excel]

Para o seu caso eu reduziria para:

```
PLATFORM_ADMIN

TENANT_ADMIN

USER
```

Nada além disso.

Problema crítico nº 4

tenant_id inconsistente

Encontramos exatamente o que eu suspeitava.

A tabela:

```
public.tenant
```

usa:

```
id INTEGER
```

como PK. [schemas_tables | Excel]

Porém quase todo o restante do sistema usa:

```
tenant_id VARCHAR
```

[schemas_tables | Excel]

Isso é muito perigoso.

Porque existem dois identificadores diferentes:

```
tenant.id

tenant.tenant_code
```

[schemas_tables | Excel]

Minha recomendação:

Padronizar definitivamente:

```
tenant_code
```

como chave lógica.

Problema crítico nº 5

Dados de conversa no schema public

Hoje temos:

```
copilot_conversations

copilot_messages
```

```
4
5  copilot_dashboards
6
7  copilot_favorites
8
9  copilot_sessions
```

todos no:

```
1  public
```

[schemas_tables | Excel]

Na arquitetura que você descreveu eles deveriam estar:

```
1  empresa_x
```

e não no schema compartilhado.

Porque:

- conversa contém contexto empresarial
- dashboard contém indicadores empresariais
- histórico contém perguntas empresariais

Recomendação:

Mover para schema do tenant.

Problema crítico nº 6

Ausência do objeto central Analysis

Eu confirmei agora no CSV.

Não existe:

```
1  copilot_analyses
```

[schemas_tables | Excel]

Mas existem:

```
1  conversations
2  messages
```

```
3  dashboards
```

[schemas_tables | Excel]

Arquiteturalmente deveria existir:

```
1  Analysis
```

no meio.

Fluxo correto:

```
1  Pergunta
2
3  ↓
4
5  Analysis
6
7  ↓
8
9  Insights
10
11  ↓
12
13  Dashboard
14
15  ↓
16
17  Resposta
```

Hoje aparentemente ocorre:

```
1  Pergunta
2
3  ↓
4
5  Dashboard
```

Problema crítico nº 7

Módulos duplicados

Encontramos:

```
protheus_modules_master
```

[schemas_tables | Excel]

e também:

```
business_domain_modules
```

com:

```
mod_code
mod_sigla
```

[schemas_tables | Excel]

A arquitetura pode ser simplificada para:

```
modules_master
```

como origem única.

O que eu faria no projeto

Manter

- ✅ public.tenant
- ✅ tenant_contracts
- ✅ tenant_module_contracts
- ✅ protheus_modules_master
- ✅ documents
- ✅ document_chunks
- ✅ business_rules
- ✅ dictionary_tables
- ✅ dictionary_fields
- ✅ business_synonyms
- ✅ audit_logs
- ✅ connectors

✅ environments

[schemas_tables | Excel]

Descontinuar

❌ tenant_table_permissions

❌ tenant_field_permissions

❌ permissions

❌ role_permissions

❌ business_field_aliases

❌ company_info

[schemas_tables | Excel]

Criar

tabela central

```
copilot_analyses
```

com:

```
analysis_id
tenant_code
conversation_id
intent_code
domain_code
module_id
analysis_json
metrics_json
insights_json
 created_at
```

Estrutura final recomendada

```
public
│
├── tenant
├── tenant_contracts
├── tenant_module_contracts
├── protheus_modules_master
├── audit_logs
├── license_plans
├── users
└── environments
```

e para cada cliente:

```
schema_cliente

dictionary_tables
dictionary_fields

business_rules

documents
document_chunks

 memories

 conversations
 messages

 analyses

 dashboards
```

Essa é a arquitetura mais aderente ao objetivo que você definiu: **um copilot analítico Protheus multiempresa, sem replicação de dados transacionais, com isolamento físico por schema e governança simplificada baseada em contrato, não em milhares de permissões individuais.** [schemas_tables | Excel]